

Robot Competition: Final Report

Robot Design and Strategy Overview

Mechanical

To maximize the number of blocks collected while fulfilling dimensional constraints, pivoting arms were designed and 3D printed with PLA to fit around two servo motors using M2 x 10 mm screws. The servo motors are oriented such that the arms rotate inward and outward, allowing the robot to meet initial size constraints while maximizing its domain when the arms pivot downward. The arms are curved to guide blocks toward the robot's interior space and are hollowed internally to reduce material cost. Additionally, the design features a cardboard block-parting wedge at the back of the chassis. The triangular shape of the wedge guides the blocks into the extended domain defined by the arms. The wedge was designed to cover the wheels, ensuring the blocks do not impact the robot's movement. Standoffs are inserted at the top of the chassis for space to place the 9V and 6V batteries in between the chassis and the Arduino UNO. To reduce the outward bending of the two arms, a piece of yarn connects the arms with the block-parting edge. A back cardboard panel secures the two cardboard pieces mounting the servo motors to the side of the chassis, further reducing bending of the arms. The two rubber bands provided were inserted around the wheels to increase traction between the wheels and the board, allowing effective movement. Two QTI sensors are screwed in under the chassis to minimize the distance between the sensor and the board, reducing ambient light interference. Similarly, the color sensor, though not electrically integrated into the system, serves as a spacer between the wedge and the wheels, preventing the wedge from interfering with the wheel's rotation.

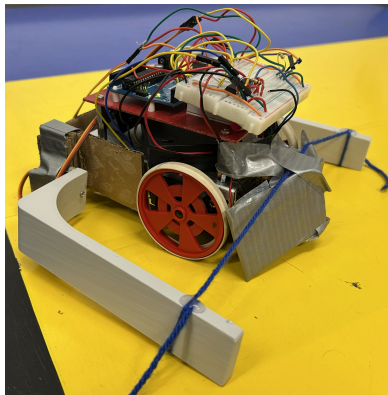
Electrical

The robot's electrical system is composed of two QTI sensors each connected in parallel with a 220 Ohm resistor, two continuous servo motors, as well as two L9110H H-bridges to implement the wheel-driving motors provided in the kit (Parallax Continuous Rotation Servo #900-00008). In addition to these components, an Arduino UNO, a 9V battery, and a 4xAA battery pack (providing 6V) are used to power and interface with the electrical system. The H-bridge is connected to four output pins of the Arduino UNO, each pin controlling the rotational direction of a motor. The 9V battery served as the primary power source of the system, supplying 9V to the Arduino UNO. The QTI sensors and servo motors are then powered from the 5V provided by the Arduino UNO. The H-bridges are powered separately using the 4xAA battery pack, allowing the wheel-driving motors to rotate at a slightly higher speed than they would if powered by the 5V provided by the Arduino UNO. The QTI sensors and servo motors both have a 3-pin connector consisting of a connection to a 5V power supply, ground, and a microcontroller I/O pin for signal input and output control. A 220 Ohm resistor is placed between the 5V power supply and signal pins of each QTI sensor to reduce sensor sensitivity.

Software

The robot begins driving towards the middle of the arena and pivoting the arms to the side. Reaching the middle, the robot turns left 90 degrees and moves in a zig-zag pattern towards the

black border to collect blocks along the middle. When it reaches the border, there are three states the robot could be in 1.) Both QTI sensors detect the black border, 2.) only the right QTI sensor detects the black border, and 3.) only the left QTI sensor detects the black border. In state 1, the robot is programmed to turn 180 degrees and continue the zig-zig algorithm. If the robot arrives at the border at an angle, it is crucial to determine how the robot should turn to prevent it from falling off the arena or turning right back into the border. As such, when it drives with the border on its right, it aims to reverse and rotate approximately 135 degrees to its left; if the border is on its left, it exhibits the same behaviour except it rotates to its right. After the bot intersects the black border three times, the robot will stop operating.



[Figure 1: Robot Chassis]

Design Process Reflection

The start of this project was focused heavily on the high-level design and algorithm for the robot. This involved sketching out a variety of designs for the robot and creating a finalized CAD. The robot was initially designed to execute a relatively simple algorithm, with the robot moving straight to the blue-yellow border, turning, and stopping at the border; algorithmic complexity was added as the project progressed. Different components of the electrical system were tested separately before final integration, aligning with the sequence of the milestones. In terms of the robot design process, there were a few significant roadblocks encountered. Firstly, larger wheels were unavailable in the lab, so that there was no longer sufficient space for the blocks to be stored within the chassis interior. Due to this roadblock, it was necessary to adjust the dimensions of the arm and wedge. Furthermore, after determining all material costs, it was decided that the arms would be the only component manufactured using 3D print; the block-parting wedge and servo plates were made of cardboard and the arms were hollowed out substantially to reduce material costs. From a hardware perspective, there were difficulties with the initial implementation of the color sensor. For a given color, a high variation in the period was reported by the sensor. As a result, it was difficult for the sensor to distinguish between the colors on the board and execute commands as intended. The color sensor was not implemented in the final electrical system of the robot due to the high fluctuation in period measurements.

Competition Analysis

The robot entered the first round with replaced batteries. Once the match began, the robot moved as expected but the servos did not rotate. This meant that nothing could collect the blocks. Afterwards, the battery voltage was tested and measured to be around 7V, which was not enough to power the whole circuit. For future rounds, the battery was replaced with the old one which had around 8.5V. In the second round, the robot could not move its wheels due to the 6V battery pack wire used to power the H-bridges falling out, which powered the two motors. The third round was the first time it performed successfully. In future rounds, the robot fell off the board and became stuck when turning in close proximity to the black border, preventing the robot from collecting more blocks. This behavior could be attributed to it not following the algorithm precisely or as expected at the border. However, this proved to be a strength, as it would collect approximately half the blocks and stop, while other robots either lost their blocks when traversing or drove off the arena. Additionally, the design and algorithm caused most of the blocks collected to congregate in one arm when making sharp turns, a preventable weakness.

Conclusions

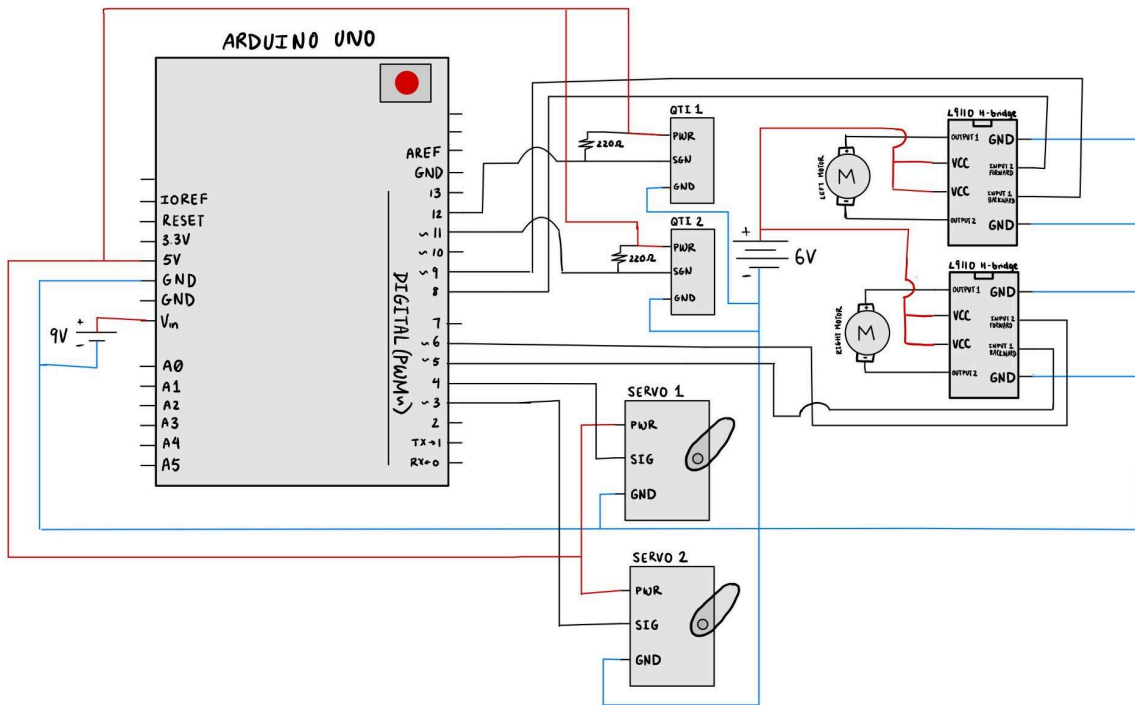
If the project were to be redone given the same constraints and budget, many changes would be made to the design and algorithm of the robot. A challenge encountered during competition was that blocks often left the robot's domain when it made sharp turns or fell off the board entirely. To ensure the blocks remained in its domain more effectively, a physical mechanism could be designed to prevent the blocks from leaving. In addition, code that ensures the robot stops at the optimal time before driving off the border and turns smoothly would allow the robot to hold existing blocks. In addition, the material costs of the robot would be calculated well in advance. Certain parts of the robot, such as the block-parting wedge, would be more secure if they were manufactured with the proper material and manufacturing process. If the high cost of RPL's 3D printing service was accounted for earlier in the design process, an improved manufacturing plan could have been sorted out for both the wedge and the mounting plate between the mini servo motors and the chassis frame. A more robust manufacturing plan would reduce the high bending the arm mechanism was subject to and allow the block-parting wedge to guide the blocks more effectively. As for the robot's electrical system, the breadboard wires should have been secured to prevent frequent shifts in wiring, reducing the repetitive debugging of the robot's wiring and disconnections. For future MAE 3780 students, it is vital that the environment of competition is simulated for testing early on in the project. For instance, the board margin was accounted for when initially testing the robot; however, it was not the board used for competition, so whenever the robot turned, it would drive off the board. As a result, the robot needed to be programmed to back up before turning. Additionally, if possible, test the robot against other robots as it is helpful towards addressing any mechanical and algorithmic issues. Finally, our team found that good collaboration is very beneficial towards completing the checkpoints and debugging the robot. Although parallel task management is important towards making effective progress, ensuring all team members understand every component of the robot is helpful towards debugging and identifying unheard of issues.

Appendix A

BILL OF MATERIALS: Mechatronics Robot									
Material	Quantity	Volume (cm³)	Mass (g)	Unit Price (\$)	Weight (lbs)	Total Price (\$)	Supplier	Robot Component	
PLA Filament	2	27.09	25.40	11.16	0.06	22.32	RPL	Pivoting arm	
Cardboard	1			2.23	0.06	2.23	McMaster	Back plate, servo mounting plate, block-parting wedge	
Yarn	1		0.00	3.89	0.00	3.89	Students Apt./Michael's	Pivoting arm addition	
Total					0.11	28.43			

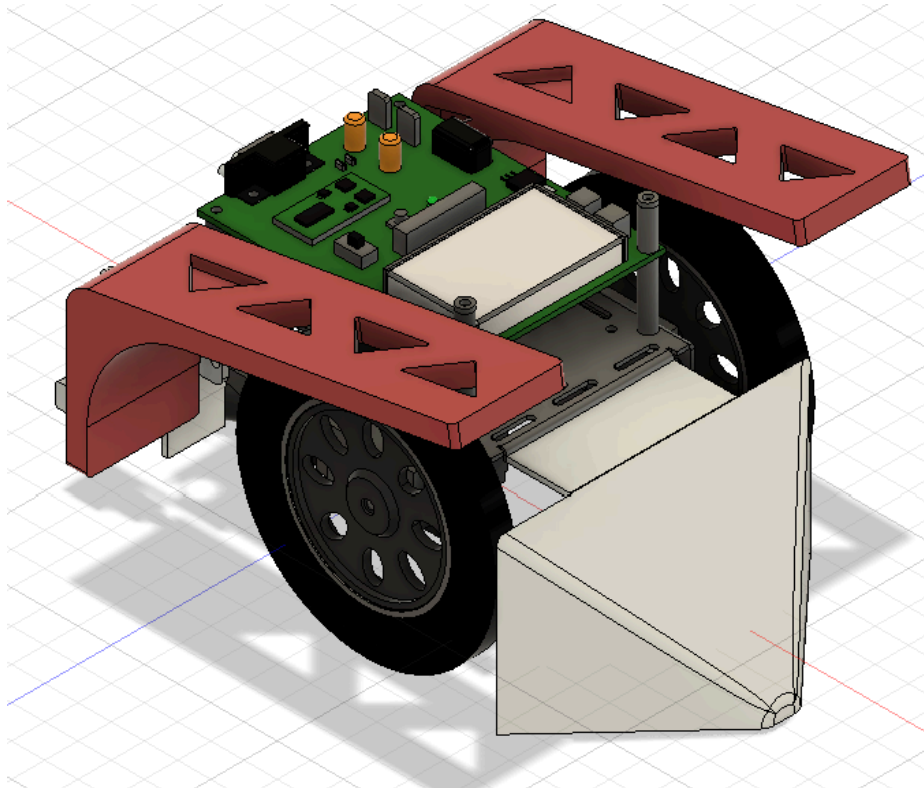
[Figure 2: Bill of Materials]

Appendix B

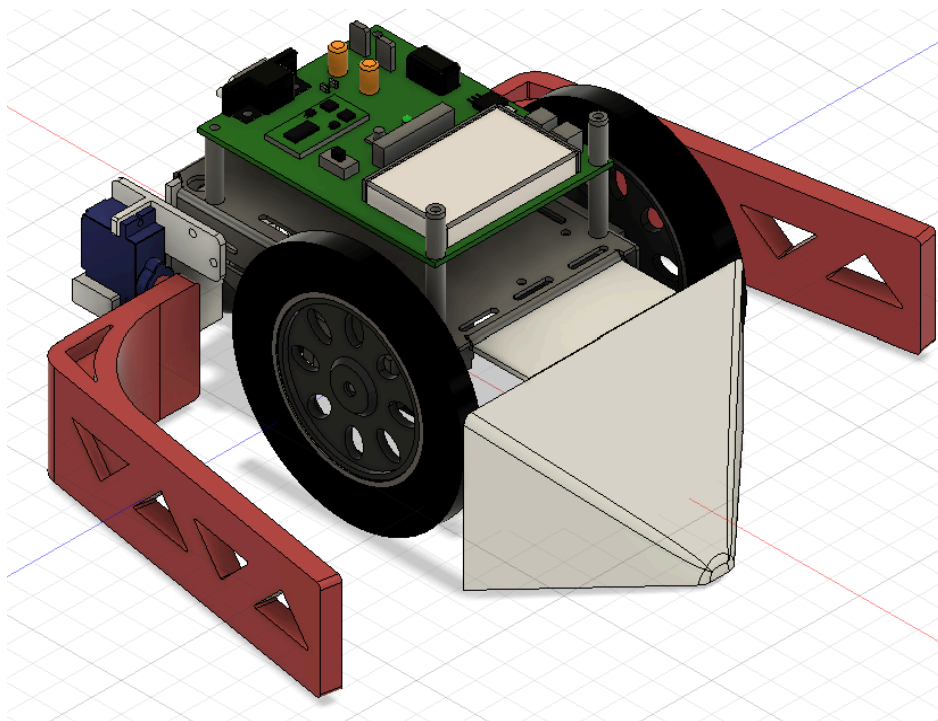


[Figure 3: Circuit Diagram]

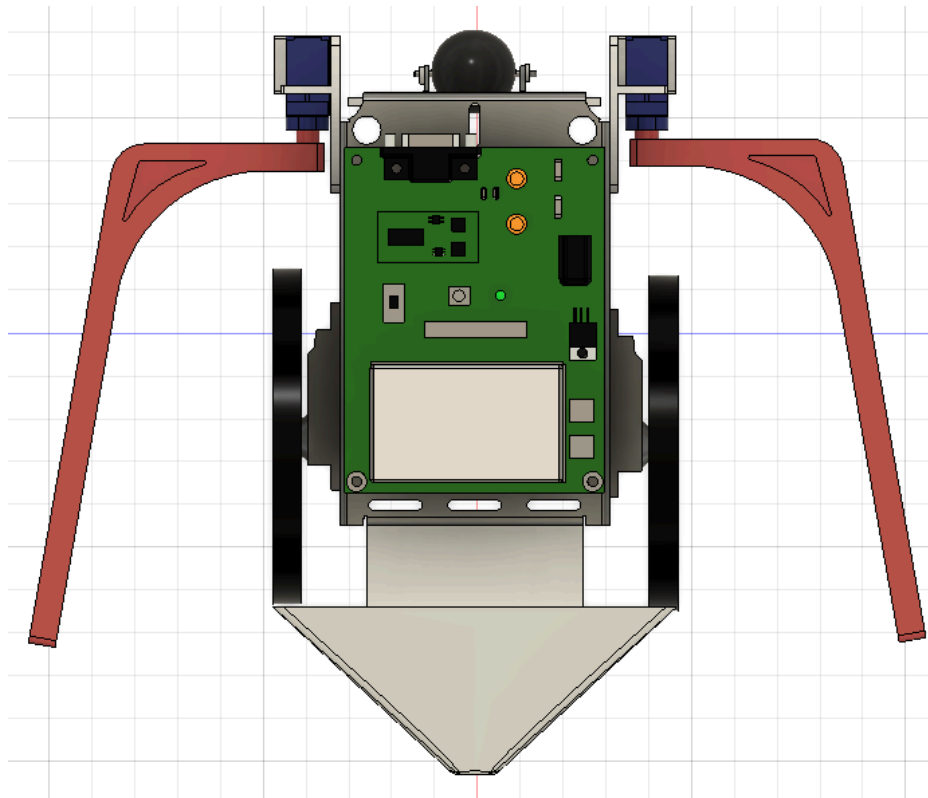
Appendix C



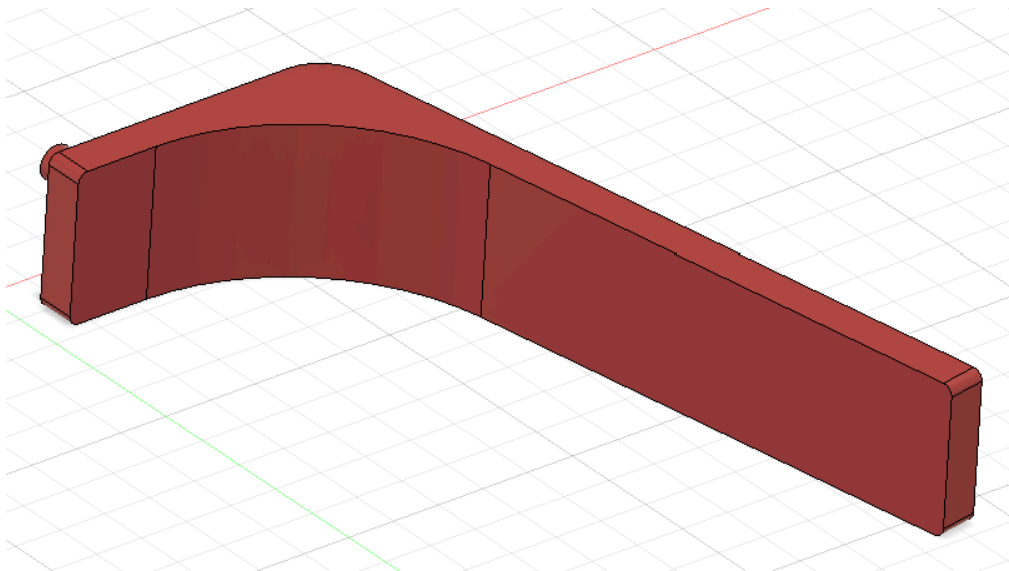
[Figure 4: Perspective-View of Initial Arm Configuration]



[Figure 5: Perspective-View of Final Arm Configuration]



[Figure 6: Top-View of Original CAD Model]



[Figure 7: Updated Robot Arm CAD Model]

Component

Pivoting Arm

x





Material Name

PLA

▼ Manage

Item Number

Lifecycle

Revision

State

Working

Change Order

▼ Physical

Mass

25.409 g

Volume

27.088 cm³

Density

0.938 g / cm³

Area

275.401 cm²

World X,Y,Z

-0.50 cm, 0.50 cm, 0.00 ...

Center of Mass

5.834 cm, 1.397 cm, 5.68...

[Figure 8: CAD of Pivoting Arm Weight]

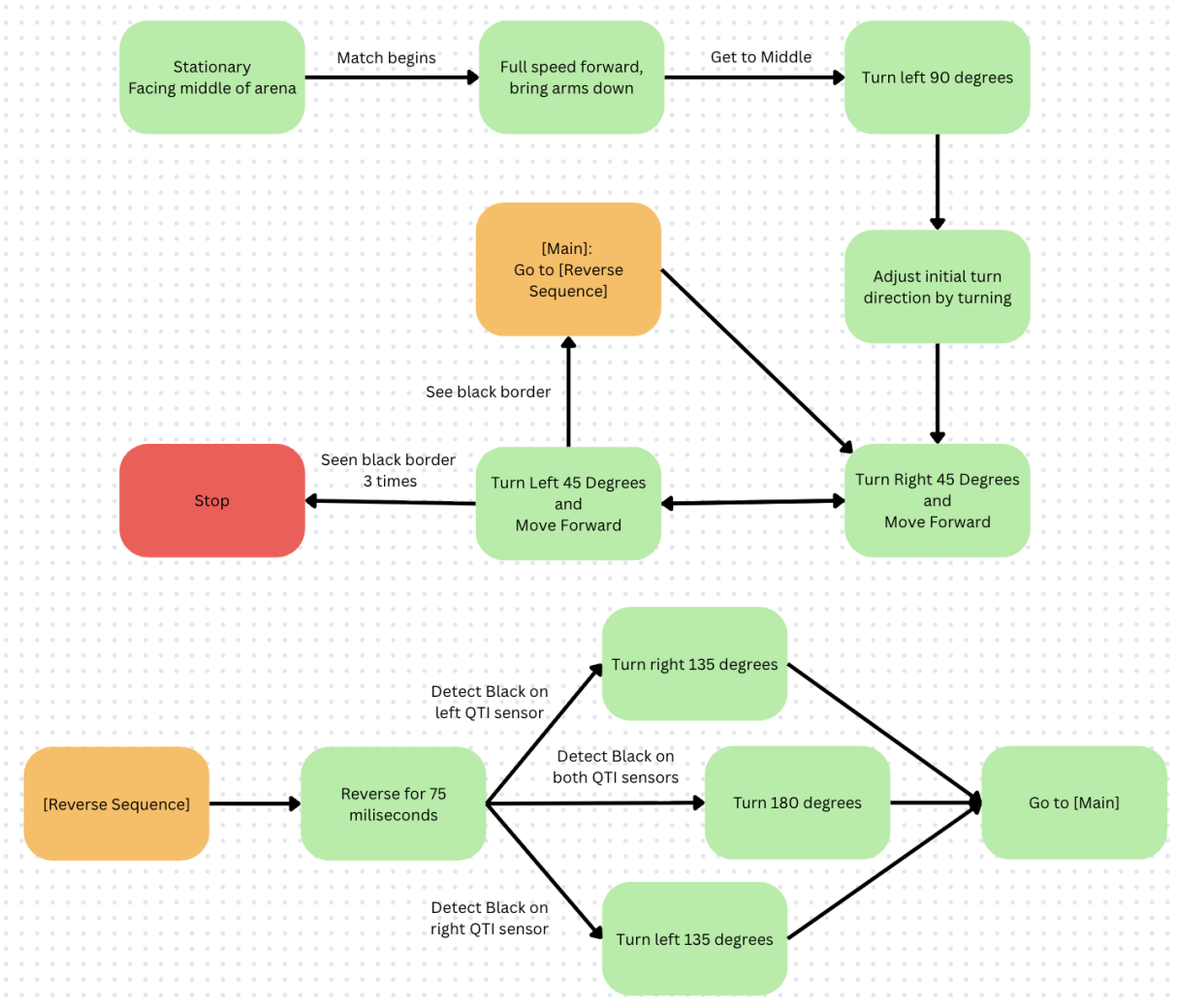


[Figure 9: Weight of Cardboard in g, 26g = 0.057lbs]



[Figure 10: Weight of Yarn in oz, 0oz = 0lbs]

Appendix D



[Figure 9: Algorithm Diagram]

Appendix E

```
// "timer" : stores the value of TIMER1
volatile float timer1 = 0;

// "left" : stores whether left QTI sensor detects black
volatile int left = 0;

// "right" : stores whether right QTI sensor detects black
volatile int right = 0;

// "turn_orientation" : stores last turn (left = 0, right = 1)
```

```
volatile int turn_orientation = 0;
// "count" : number of iterations
volatile int count = 3;

// black intensity threshold
#define BLACK_THRESHOLD 4900

// Left motor pins
int pin8 = 0; // PINB0
int pin9 = 1; // PINB1

// Right motor pins
int pin5 = 5; // PIND5
int pin6 = 6; // PIND6

// QTI pins
int left_qti = 3; // PIND3
int right_qti = 2; // PIND2

// SERVO pins
#define SERVO1_PIN 4 // PORTB bit 4
#define SERVO2_PIN 5 // PORTB bit 5

// Forward declarations
void initQTI();
int readQTI(int pin);
void drive_forward();
void drive_backward();
void drive_left();
void drive_right();
void stop();
```

```

void drive_delay(int t);

// send one PWM pulse to a servo
void servoPulse(uint8_t pin, int pulse_us) {
    PORTB |= (1 << pin); // HIGH
    _delay_us(pulse_us); // hold for pulse width
    PORTB &= ~(1 << pin); // LOW
    _delay_us(20000 - pulse_us); // remainder of 20ms period
}

// send pulses for a given duration (ms) at a given pulse width
void servoRun(uint8_t pin, int pulse_us, int duration_ms) {
    // Each cycle is 20ms, so calculate how many cycles fit in duration
    int cycles = duration_ms / 20;
    for (int i = 0; i < cycles; i++) {
        servoPulse(pin, pulse_us);
    }
}

// return QTI output
int readQTI(int pin) {
    DDRD |= (1 << pin); // set as output
    PORTD |= (1 << pin); // drive HIGH
    _delay_ms(1);

    DDRD &= ~(1 << pin); // set as input
    PORTD &= ~(1 << pin); // disable pullup

    int count = 0;
    while (PIND & (1 << pin)) {
        count++;
    }
}

```

```

    if (count > 5000) break;
}
return count;
}

int main(void) {
    Serial.begin(9600);
    // inits
    DDRB = 0b00110011;
    DDRD = 0b01100000;
    PORTB = 0b0;
    PORTD = 0b0;
    sei();
    // rotate servo 1 approximately 90 degrees
    #define ROTATE_MS 500 // start here and tune up/down

    int i = 0;
    int turn_orientation = 0;

    // going to the middle
    drive_forward();
    // drive the arms down
    servoRun(SERVO2_PIN, 1750, ROTATE_MS);
    servoRun(SERVO1_PIN, 2000, ROTATE_MS);
    drive_delay(2000);
    // reach the middle and turn left
    drive_left();
    drive_delay(650);
    stop();
    int initial = 0;
    while (1){

```

```

/*
Reads QTI sensors and determines
whether the QTIs detect black
*/

int leftVal = readQTI(left_qti);
int rightVal = readQTI(right_qti);
left = (leftVal > BLACK_THRESHOLD);
right = (rightVal > BLACK_THRESHOLD);

// Serial.print("L: "); Serial.print(leftVal);
// Serial.print(" R: "); Serial.print(rightVal);
// Serial.print(" i: "); Serial.println(i);

// stop the robot when it detects the black border 3 times
if (i > count){
    stop();
    break;
}
if (left || right){ //detected black
    stop();
    drive_backward();
    drive_delay(75);
    if (left && right){ //detect black on both QTI sensors
        // Serial.print("Detected black turn 180 ");
        drive_left();
        drive_delay(1000);
        drive_forward();
        left = 0;
        right = 0;
    } else if (left){ //detect black on left QTI sensor
        // Serial.print("Detected black turn RIGHT ");

```

```

    drive_right();
    drive_delay(750);
    stop();
    left = 0;
} else if (right) { //detect black on right QTI sensor
    // Serial.print("Detected black turn LEFT ");
    drive_left();
    drive_delay(750);
    stop();
    right = 0;
}
i += 1;
initial = 0;
continue;
}
//zig zag
if (!initial) { //adjust the turn direction
    //initial turn
    drive_right();
    drive_delay(345);
    stop();
    turn_orientation = 1;
    initial = 1;
}
drive_forward();
drive_delay(500);
if (!turn_orientation) { //last turned left
    drive_right();
    drive_delay(500);
    stop();
    turn_orientation = 1;
}

```



```

    } else{ //last turned right
        drive_left();
        drive_delay(500);
        stop();
        turn_orientation = 0;
    }
}
}

// left QTI sensor detected black
ISR(INT0_vect) {
    left = 1;
}

// right QTI sensor detected black
ISR(INT1_vect) {
    right = 1;
}

// initialize QTI
void initQTI() {
    DDRD &= ~(1 << left_qti);
    DDRD &= ~(1 << right_qti);
}

// drive robot backwards
void drive_backward() {
    PORTB |= (1 << pin9);
    PORTD |= (1 << pin5);
}

// drive robot forwards
void drive_forward() {
    PORTB |= (1 << pin8);

```

```
PORTD |= (1 << pin6);
}
// drive robot left
void drive_left() {
    stop();
    PORTB |= (1 << pin8);
    PORTD |= (1 << pin5);
}
// drive robot right
void drive_right() {
    stop();
    PORTB |= (1 << pin9);
    PORTD |= (1 << pin6);
}

// stop the robot
void stop() {
    PORTB &= ~(1 << pin8);
    PORTB &= ~(1 << pin9);
    PORTD &= ~(1 << pin5);
    PORTD &= ~(1 << pin6);
    _delay_ms(100);
}

//drive delay
void drive_delay(int t) {
    _delay_ms(t);
}
```